

Práctica del Laboratorio 3

Arquitectura del Computador

5 de Agosto de 2026

1. Explique cómo se organiza la memoria cuando un sistema utiliza memory-mapped I/O. ¿En qué región de memoria se suelen mapear los dispositivos? ¿Qué implicaciones tiene para las instrucciones lw y sw?

Memory-mapped I/O es una técnica donde los registros de los dispositivos de E/S se asignan a direcciones del espacio de memoria del procesador, como si fueran celdas de memoria normales.

Los dispositivos de E/S se mapean típicamente en la parte alta de la memoria (ej. 0xFFFF0000 en MIPS32), reservando esa zona para los periféricos y dejando el resto para la RAM.

Implicaciones para lw y sw: La gran ventaja es que usamos las mismas instrucciones de siempre para leer y escribir en los dispositivos. Cuando hacemos un lw o sw con una dirección como 0xFFFF0004, el hardware se da cuenta de que es una dirección de E/S y en lugar de ir a la RAM, se comunica directamente con el sensor. Es como si el procesador pensara que está hablando con memoria normal, pero por debajo está hablando con el hardware.

2. ¿Cuál es la principal diferencia entre memory-mapped I/O y la entrada/salida por puertos? ¿Qué ventajas y desventajas tiene cada enfoque? ¿Por qué MIPS32 utiliza principalmente memory-mapped I/O?

En memory-mapped I/O, los dispositivos se ven como celdas de memoria normales y se accede con lw y sw. En entrada/salida por puertos, se usan instrucciones especiales como in y out para comunicarse con los periféricos, teniendo un espacio de direcciones separado para E/S.

Memory-Mapped I/O	Entrada/Salida por Puer- tos
Ventajas	
Usa las mismas instrucciones que la memoria (lw/sw)	Espacio de direcciones separado (no ocupa memoria RAM)
Más simple y flexible	No hay problemas con caché
Soporta modos de direccionamiento complejos	-
Desventajas	
Ocupa espacio de direcciones que podría usar la RAM	Necesita instrucciones especiales
Problemas con la caché	Más complejo de programar

¿Por qué MIPS32 usa memory-mapped I/O? Porque MIPS32 es un procesador RISC y su filosofía es tener un conjunto de instrucciones pequeño y simple. En lugar de añadir instrucciones especiales para E/S, prefieren usar las que ya tienen (lw/sw) y mapear los dispositivos en memoria. Así el hardware es más sencillo, el compilador más fácil de implementar y el acceso a periféricos es más uniforme y consistente.

3. En un sistema con memory-mapped I/O: ¿Qué problemas pueden surgir si dos dispositivos usan direcciones solapadas? ¿Cómo se evita este conflicto?

Si dos dispositivos usan la misma dirección, ambos intentan responder a la vez, causando lecturas corruptas y escrituras accidentales. Se evita con un mapa de memoria fijo donde cada dispositivo tiene un rango de direcciones exclusivo y sin solapamiento, y el hardware decodifica la dirección para activar solo el periférico correspondiente.

El mapa de memoria fijo es una tabla que asigna rangos de direcciones exclusivos a cada dispositivo y a la RAM. El hardware sabe que, por ejemplo, 0x00000000 - 0x7FFFFFFF es para RAM y 0xFFFF0000 - 0xFFFFFFFF es para periféricos. Cuando el procesador genera una dirección, la lógica de decodificación activa solo el dispositivo dueño de ese rango, evitando conflictos.

4. ¿Por qué se considera que el memory-mapped I/O simplifica el diseño del conjunto de instrucciones de un procesador? ¿Qué tipo de instrucciones adicionales serían necesarias si se usara E/S por puertos?

Memory-mapped I/O simplifica el diseño porque no se necesitan instrucciones especiales para E/S. El procesador solo necesita su conjunto básico de instrucciones de memoria (lw, sw, lb, sb, etc.) para comunicarse con los periféricos. Esto hace que el hardware sea más sencillo, el decodificador de instrucciones más pequeño y el compilador más fácil de implementar.

Instrucciones adicionales para E/S por puertos: Si se usara E/S por puertos, se necesitarían instrucciones especiales como:

- **in (input):** Lee un dato desde un puerto de entrada.
- **out (output):** Escribe un dato en un puerto de salida.
- **ins (input string):** Lee bloques de datos desde puertos.
- **outs (output string):** Escribe bloques de datos a puertos.

Estas instrucciones requieren un espacio de direcciones separado, lógica adicional para el decodificador y más complejidad en el compilador.

5. ¿Qué ocurre a nivel del bus de datos y direcciones cuando el procesador accede a una dirección de memoria que corresponde a un dispositivo? ¿Cómo sabe el hardware que debe acceder a un periférico en lugar de la RAM?

Cuando el procesador ejecuta un lw o sw a una dirección de E/S (ej. 0xFFFF0004):

1. El procesador coloca la dirección en el bus de direcciones.
2. La lógica de decodificación (chip select) detecta que la dirección está en el rango reservado para E/S (ej. 0xFFFF0000 - 0xFFFFFFFF).
3. En lugar de activar la señal de habilitación de la RAM, se activa la señal correspondiente al periférico (ej. el sensor de tensión). Esto se logra mediante un decodificador que compara los bits más significativos de la dirección y genera una señal específica para cada dispositivo.
4. Los datos fluyen entre el procesador y el dispositivo a través del bus de datos. Si es un sw, el procesador coloca el dato en el bus de datos y el periférico lo captura. Si es un lw, el periférico coloca el dato en el bus y el procesador lo lee.

El hardware sabe que debe acceder al periférico en lugar de la RAM porque la decodificación de direcciones reconoce el rango asignado a E/S y genera la señal de control adecuada para el dispositivo específico. Así, el procesador no necesita saber si está accediendo a memoria o a un periférico; simplemente ejecuta la instrucción y el hardware se encarga del resto.

6. ¿Es posible que un programa normal (sin privilegios) acceda a un dispositivo mapeado en memoria? ¿Qué mecanismos de protección existen para evitar accesos no autorizados?

En el simulador MARS/SPIM que usamos en clase, sí es posible porque no hay sistema operativo dentro de la simulación y todo el código se ejecuta en modo kernel simulado, por

lo que podemos usar lw y sw a cualquier dirección sin restricciones. En un sistema MIPS real con Linux, un programa normal sin privilegios no puede acceder directamente a dispositivos mapeados en memoria porque el hardware y el sistema operativo implementan mecanismos de protección.

Mecanismos de protección para evitar accesos no autorizados Los principales mecanismos son los siguientes:

- **Modos de operación del procesador (usuario y kernel):** Los accesos a direcciones de E/S solo están permitidos en modo kernel; si un programa en modo usuario intenta acceder, el procesador genera una excepción.
- **MMU (Unidad de Gestión de Memoria):** Traduce direcciones virtuales a físicas y marca las páginas de E/S como inaccesibles para programas de usuario.
- **Sistema operativo:** Captura las excepciones generadas por accesos no autorizados y normalmente termina el proceso.

En los sistemas reales, los programas de usuario acceden a los dispositivos a través de drivers y llamadas al sistema, donde el kernel es el único que tiene acceso directo a las direcciones de E/S y realiza las operaciones de forma segura.

7. ¿Qué técnicas se pueden emplear para evitar esperas activas innecesarias al interactuar con dispositivos?

La espera activa (polling) consiste en que el procesador consulte constantemente el estado del dispositivo para saber si ya terminó, lo cual es ineficiente porque desperdicia ciclos que podrían usarse en otras tareas.

Para evitarlo, se utilizan principalmente dos técnicas:

- **Interrupciones:** Permiten que el dispositivo notifique al procesador cuando está listo, sin necesidad de que el procesador esté preguntando continuamente. Cuando ocurre una interrupción, el procesador pausa su ejecución, atiende al dispositivo mediante un manejador de interrupción, y luego continúa con su tarea anterior.
- **DMA (Direct Memory Access):** Utiliza un controlador especial para transferir datos entre el dispositivo y la memoria sin intervención del procesador. El procesador solo configura la transferencia (origen, destino, tamaño) y puede dedicarse a otras tareas; al finalizar, el DMA genera una interrupción para avisar al procesador.

Estas técnicas mejoran el rendimiento porque el procesador no queda bloqueado esperando al dispositivo.

8. Análisis y Discusión de los Resultados

1. Sensor de Luz El código implementa un controlador para un sensor de luminosidad usando memory-mapped I/O. InicializarSensorLuz escribe 1 en LuzControl y espera activamente hasta que LuzEstado sea 1 (listo) o -1 (error). LeerLuminosidad verifica que el estado sea 1, lee el valor de LuzDatos y lo guarda en \$s0, retornando 0 si éxito o -1 si error. El uso de \$s0 es correcto porque es un registro preservado. La espera activa es simple pero ineficiente, y en sistemas reales se reemplazaría por interrupciones.

2. Controlador de Tensión Arterial controlador_tension escribe 1 en TensionControl y espera hasta que TensionEstado sea 1. Luego lee TensionSistol y TensionDiastol, guardándolos en \$v0 y \$v1. El main imprime ambos valores separados por un espacio. El código es funcional pero, al igual que el sensor de luz, la espera activa desperdicia ciclos del procesador. La principal diferencia con el sensor de luz es que este retorna dos valores en lugar de uno.

Conclusión

Los códigos cumplen con los requisitos planteados y son correctos a nivel funcional. Demuestran cómo memory-mapped I/O en MIPS32 permite acceder a periféricos usando instrucciones estándar como lw y sw, sin necesidad de instrucciones especiales. Las direcciones de los registros están en la parte alta de la memoria, siguiendo el mapa del sistema. La espera activa es adecuada para fines educativos pero ineficiente en sistemas reales, donde se usarían interrupciones. En general, ambos códigos cumplen con los requisitos y sirven como base para entender la interacción entre el procesador y los dispositivos de E/S.